# Solution: Multiply Strings

## Problem Statement

Given two non-negative numbers `num1` and `num2` represented as strings, return the multiplication of `num1` and `num2` , represented as a string.

### Examples

- **Example 1**:
  - **Input**: num1: `4`, num2: `5`
  - **Expected Output**: "**20**"
  - **Justification**: Multiplying 4 by 5 gives 20, showcasing simple multiplication.
- **Example 2**:
  - **Input**: num1: `11`, num2: `11`
  - **Expected Output**: "**121**"
  - **Justification**: The multiplication of 11 by 11 equals 121.
- **Example 3**:
  - **Input**: num1: `999`, num2: `999`
  - **Expected Output**: "**998001**"
  - **Justification**: The multiplication of 999 by 999 equals 998001.

## Solution

To solve this problem, the key is to mimic the traditional multiplication method we use by hand, where numbers are multiplied digit by digit and the intermediate results are added to get the final product. This approach is effective because it allows us to handle very large numbers that might not fit into standard data types. By iterating over each digit of both numbers starting from the least significant digit, we can ensure that every digit is accounted for in the multiplication process. The product of each pair of digits is placed at the correct position in the result array to handle carry and accumulation accurately.

This strategy is chosen because it does not rely on converting strings to integers, thus avoiding overflow or precision issues with very large numbers. It directly manipulates the strings and uses basic arithmetic principles to achieve the result. This method is both scalable and precise, making it the most suitable for our problem.

### Step-by-Step Algorithm

1. Initialize a result array of size `num1.length + num2.length` to hold the multiplication results, filled initially with zeros.
2. Traverse both `num1` and `num2` from right to left (i.e., from the least significant digit to the most significant).
3. For each digit in `num1` (indexed by `i`) and `num2` (indexed by `j`):
   - Multiply the digits at the current indices: `(num1[i] - '0') * (num2[j] - '0')`.
   - Calculate the position in the result array where this product should be added: `position1 = i + j` and `position2 = i + j + 1`.
   - Add the product to `result[position2]`, and manage any carry that results from this addition.
   - If there's a carry, add it to `result[position1]`.
4. Once all digits have been processed, the `result` array will contain numbers that might be more than 9 (except the last digit). Starting from the right, convert these to a single digit and carry over the remainder to the next left position.
5. Finally, convert the `result` array to a string. If the string has leading zeros, remove them before returning the result.

### Algorithm Walkthrough

Let's consider the input, num1: "999", num2: "999"

The result array has a length of `num1.length + num2.length = 6`, initially filled with zeros: `[0, 0, 0, 0, 0, 0]`.

#### **Iteration 1**: i = 2, j = 2 (Both last digits)

- **Calculation**: 9 * 9 = 81
- **Placement**: Add 81, result becomes `[0, 0, 0, 0, 8, 1]`.

#### **Iteration 2**: i = 2, j = 1 (Last digit of num1, second last of num2)

- **Calculation**: 9 * 9 = 81
- **Placement with carry**: Add to positions 3 and 4, considering carry from the previous step.
- **Result Array**: `[0, 0, 0, 8, 9, 1]`.

#### **Iteration 3**: i = 2, j = 0 (Last digit of num1, first digit of num2)

- **Calculation**: 9 * 9 = 81
- **Placement with carry**: Add to positions 2 and 3, considering carry from the previous step.
- **Result Array**: `[0, 0, 8, 9, 9, 1]` (carries will be adjusted later).

#### **Iteration 4**: i = 1, j = 2 (Second last digit of num1, last digit of num2)

- **Calculation**: 9 * 9 = 81
- **Placement with carry**: Considering previous carries, add to positions 3 and 4.
- **Result Array**: `[0, 0, 8, 18, 0, 1]`.

#### **Iteration 5**: i = 1, j = 1 (Second last digits of both numbers)

- **Calculation**: 9 * 9 = 81
- **Placement with carry**: Add to positions 2 and 3.
- **Result Array**: `[0, 0, 17, 9, 0, 1]`.

#### **Iteration 6**: i = 1, j = 0 (Second last digit of num1, first digit of num2)

- **Calculation**: 9 * 9 = 81
- **Placement with carry**: Add to positions 1 and 2.
- **Result Array**: `[ 0, 9, 8, 9, 0, 1 ]`.

#### **Iteration 7**: i = 0, j = 2 (First digit of num1, last digit of num2)

- **Calculation**: 9 * 9 = 81
- **Placement with carry**: Add to positions 2 and 3.
- **Result Array**: `[ 0, 9, 17, 0, 0, 1 ]`.

#### **Iteration 8**: i = 0, j = 1 (First digits of num1, second last of num2)

- **Calculation**: 9 * 9 = 81
- **Placement with carry**: Add to positions 1 and 2.
- **Result Array**: `[ 0, 18, 8, 0, 0, 1 ]`.

#### **Iteration 9**: i = 0, j = 0 (Both first digits)

- **Calculation**: 9 * 9 = 81
- **Placement with carry**: Add to positions 0 and 1.
- **Result Array**: `[ 9, 9, 8, 0, 0, 1 ]`.

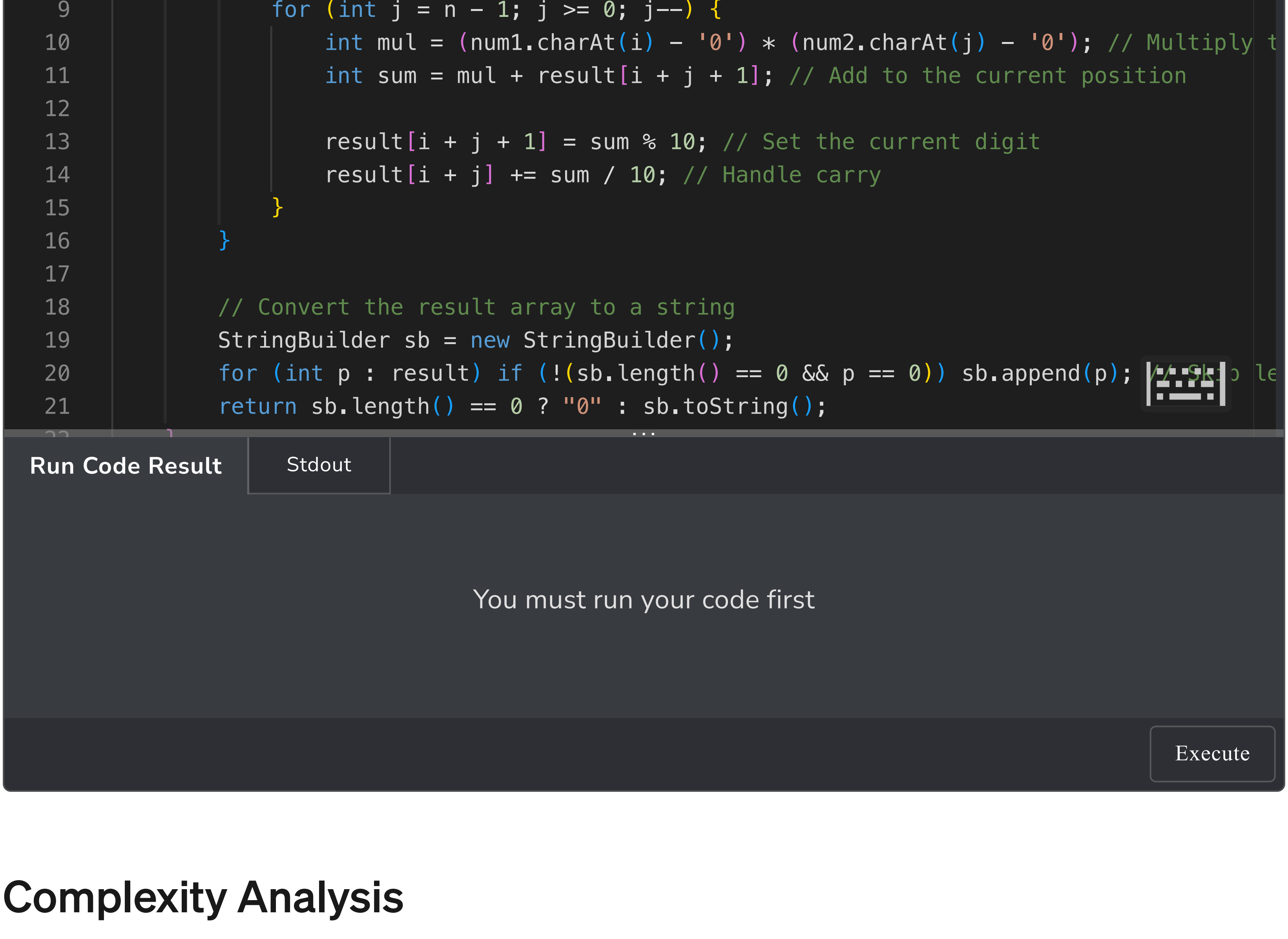
**Converting to String**:

- Convert `result` array into a string `"998001"` and remove any leading zeros if present (not required in this case since the result does not start with zero).

**Final Output**:

- The multiplication of "999" by "999" yields `"998001"` after adjusting the carries correctly in each step.

## Code



```java
public class Solution {
    // Method to multiply two strings and return the product as a string
    public String multiplyStrings(String num1, String num2) {
        int m = num1.length(), n = num2.length();
        int[] result = new int[m + n]; // Result can have at most m + n digits

        // Multiply each digit of num1 with each digit of num2
        for (int i = m - 1; i >= 0; i--) {
            for (int j = n - 1; j >= 0; j--) {
                int mul = (num1.charAt(i) - '0') * (num2.charAt(j) - '0'); // Multiply t
                int sum = mul + result[i + j + 1]; // Add to the current position

                result[i + j + 1] = sum % 10; // Set the current digit
                result[i + j] += sum / 10; // Handle carry
            }
        }

        // Convert the result array to a string
        StringBuilder sb = new StringBuilder();
        for (int p : result) if (!(sb.length() == 0 && p == 0)) sb.append(p); // Skip le
        return sb.length() == 0 ? "0" : sb.toString();
```

## Complexity Analysis

- **Time Complexity**: $O(m * n)$, where m and n are the lengths of the two input strings. This is because each digit of both numbers needs to be multiplied with every digit of the other number.
- **Space Complexity**: $O(m + n)$, required for the result array to store intermediate values. This is the maximum length the result can have.

← Previous
Multiply Strings (Medium)

Next →
Best Time To Buy And Sell Stock II (Medium)

☐ Mark as Completed